Aprendizaje de Máquina
Sesión 19: Aprendizaje de máquina para textos

Angel Rodríguez

Bag of Words

Una representación sencilla es el modelo

Bag of Words

Cada dimensión del vector corresponde a una palabra del vocabulario.

Si

$$V = [\text{machine}, \text{learning}, \text{is}, \text{useful}, \text{data}],$$

entonces

"machine learning is useful"

puede representarse como

$$x = [1, 1, 1, 1, 0].$$

¿Todas las palabras deberían pesar igual?

Supongamos que una palabra aparece muchas veces en **todos** los documentos.

¿Es realmente útil para distinguirlos?

Por ejemplo:

the, is, of, and, . . .

Queremos asignar mayor importancia a términos que

aparecen en un documento	pero no en todos los documentos

Esto motiva la representación

tf-idf

Term Frequency (TF)

La primera parte mide qué tan frecuente es una palabra dentro de un documento.

$$\mathrm{tf}_{t,d}$$

Una opción sencilla sería usar directamente la frecuencia de la palabra t en el documento d

$$\mathrm{tf}_{t,d} = \mathrm{count}(t, d).$$

Pero se utiliza una transformación logarítmica:

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t, d), & \mathrm{count}(t, d) > 0, \\ 0, & \text{en otro caso.} \end{cases}$$

Inverse Document Frequency (IDF)

Sea $N =$ número total de documentos y
$df_t =$ número de documentos que contienen el término t.
Entonces

$$\boxed{\text{idf}_t = \log_{10}\left(\frac{N}{df_t}\right)}$$

- Si t aparece en muchos documentos:

$$df_t \text{ grande} \quad \Rightarrow \quad idf_t \text{ pequeño.}$$

- Si t aparece en pocos documentos:

$$df_t \text{ pequeño} \quad \Rightarrow \quad idf_t \text{ grande.}$$

TF-IDF

Combinamos ambas cantidades:

$$\boxed{\text{tf-idf}(t, d) = \text{tf}_{t,d}\, \text{idf}_t}$$

La representación de un documento se convierte entonces en

$$x_d = [\text{tf-idf}(t_1, d), \ldots, \text{tf-idf}(t_p, d)].$$

Una palabra recibe un peso grande cuando

es frecuente en el documento y poco frecuente en la colección

Comparando documentos

Una vez que cada documento es un vector,

$$d_1 \longrightarrow x_1, \qquad d_2 \longrightarrow x_2,$$

podemos medir qué tan similares son.

Una de las medidas más utilizadas es

$$\boxed{\cos(x_1, x_2) = \frac{x_1^\top x_2}{\|x_1\|\|x_2\|}}$$

Es decir, el producto punto **normalizado por la longitud de los vectores**.

Word2Vec: idea general

Word2Vec aprende automáticamente embeddings a partir de los **contextos** en los que aparecen las palabras.

La idea general es:

Corpus $\longrightarrow$ contextos observados $\longrightarrow$ vectores aprendidos

Palabras utilizadas en contextos similares terminan con vectores cercanos en el espacio.

El significado aparece como una **relación geométrica**.

¿Qué está aprendiendo Word2Vec?

Skip-gram mantiene realmente **dos vectores por palabra**:

$$\boxed{w = \text{embedding como palabra objetivo}}$$

y

$$\boxed{c = \text{embedding como palabra de contexto.}}$$

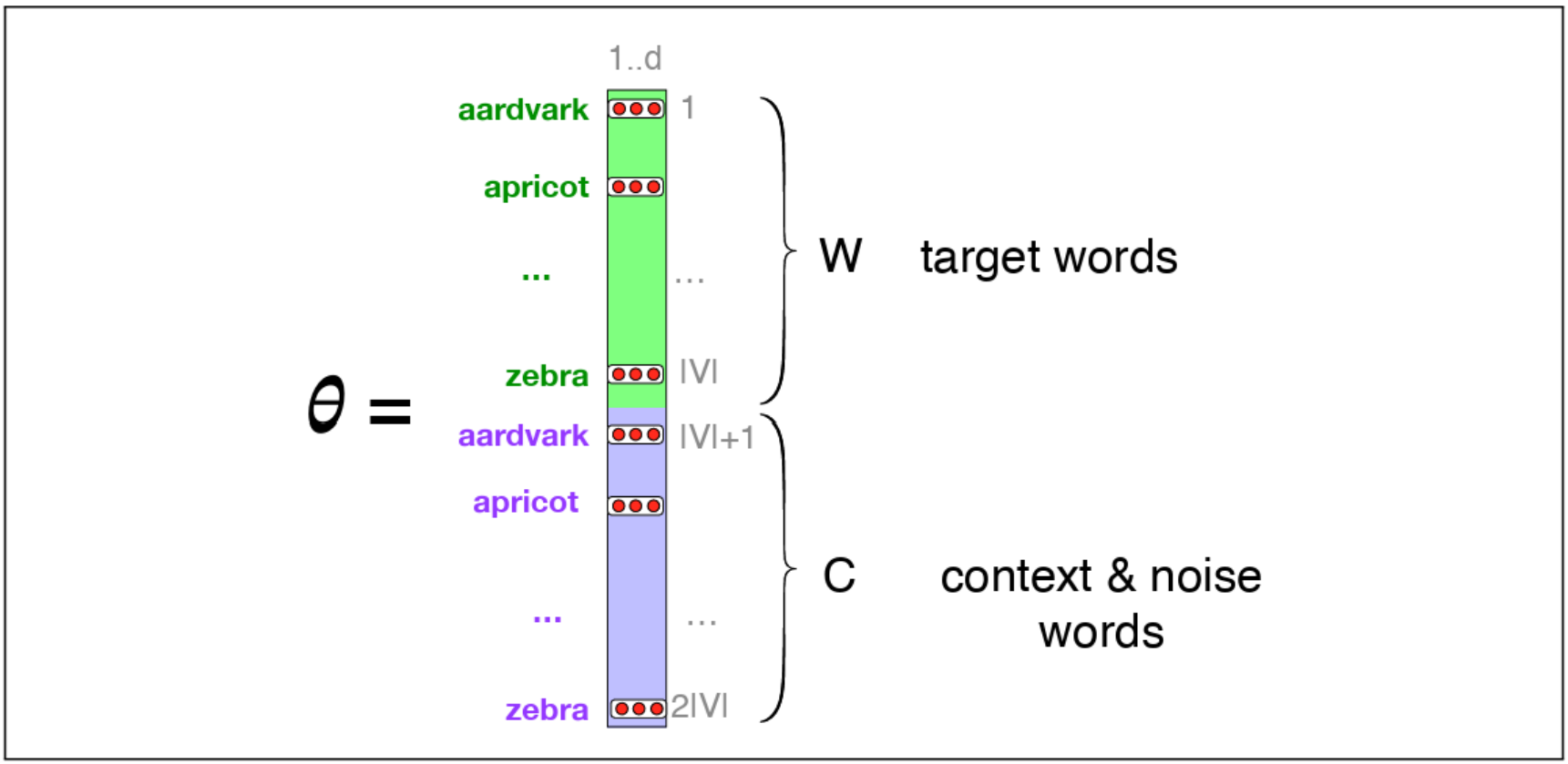
Por tanto, se aprenden dos matrices:

$$W \in \mathbb{R}^{|V| \times d}, \qquad C \in \mathbb{R}^{|V| \times d}.$$

W : embeddings de palabras objetivo
C : embeddings de palabras de contexto

Los valores de estas matrices son los **parámetros que se ajustan durante el entrenamiento**.

Skip-Gram


1..d
aardvark
1
apricot
...
...
zebra
|V|
θ =
aardvark
|V|+1
apricot
...
...
zebra
2|V|
W
target words
C
context & noise words

Entrenamiento de los embeddings

El modelo Skip-gram mantiene dos matrices:

$$W = \begin{pmatrix} --- & w_1^T & --- \\ & \vdots & \\ --- & w_{|V|}^T & --- \end{pmatrix}, \qquad C = \begin{pmatrix} --- & c_1^T & --- \\ & \vdots & \\ --- & c_{|V|}^T & --- \end{pmatrix}.$$

1. Inicializar aleatoriamente las matrices W y C.
2. Seleccionar un par positivo (w, c_{pos}) y sus k negativos.
3. Calcular la pérdida y sus gradientes.
4. Actualizar los vectores involucrados mediante las ecuaciones anteriores.
5. Repetir el procedimiento recorriendo las palabras del corpus.

Representación final de las palabras

Una vez entrenado el modelo, podemos representar la palabra i mediante:

$$\boxed{e_i = w_i + c_i}$$

o utilizar solamente el vector central w_i.

Word2Vec aprende embeddings estáticos

Una característica importante de Word2Vec es que aprende **un único vector por palabra**.

$$\boxed{w \longrightarrow w}$$

Por ejemplo,

$$\text{bank} \longrightarrow w_{\text{bank}}.$$

El vector es el mismo independientemente del contexto:

"deposit money in the bank"

"sit on the river bank".

Limitación

Word2Vec no genera una representación diferente para cada significado o contexto de una palabra.

Modelos posteriores utilizan **embeddings contextuales**, donde la representación depende de la oración en la que aparece la palabra.

Otros embeddings estáticos

Word2Vec no es el único método para obtener embeddings estáticos.

FastText
Representa una palabra utilizando también sus *n*-gramas de caracteres.
Por ejemplo, con $n = 3$:

$$\text{where} \longrightarrow \{<\texttt{wh}, \texttt{whe}, \texttt{her}, \texttt{ere}, \texttt{re}>\}.$$

Esto permite representar mejor palabras raras o incluso **palabras no observadas durante el entrenamiento**.

GloVe
Aprende embeddings utilizando estadísticas globales de **coocurrencia de palabras** en el corpus.

Word2Vec, FastText y GloVe producen **embeddings estáticos**.

Word2Vec: resumen

Corpus → pares palabra–contexto → clasificación → embeddings

1. Elegimos una palabra objetivo w.
2. Las palabras cercanas generan ejemplos positivos.
3. Muestreamos palabras aleatorias como ejemplos negativos.
4. Comparamos los embeddings mediante el producto punto.
5. Ajustamos los vectores para distinguir ambos casos.

El resultado es una representación

$$w \longrightarrow \mathbf{w} \in \mathbb{R}^d$$

densa, de baja dimensión y capaz de capturar
relaciones semánticas entre palabras.

Resumen

Para aplicar aprendizaje de máquina a textos necesitamos

convertir texto en números

Dos grandes familias:

$$\text{Texto} \rightarrow \text{BoW} \rightarrow \text{TF-IDF}$$

representaciones dispersas

$$\text{Texto} \rightarrow \text{contextos} \rightarrow \text{embeddings}$$

representaciones densas

En ambos casos obtenemos vectores que podremos utilizar para

comparar, agrupar y clasificar textos.

Siguiente paso: agrupamiento de textos

Ahora podemos representar cada documento como un vector:

$$d_i \longrightarrow x_i.$$

Por tanto, los métodos de aprendizaje no supervisado que ya conocemos pueden aplicarse directamente:

$$\{x_1, \ldots, x_n\} \xrightarrow{K\text{-means}} \text{grupos de documentos}$$

4.3 Agrupamiento de textos

En el tema anterior transformamos cada documento en un vector:

$$d_i \longrightarrow x_i.$$

Por ejemplo, utilizando TF-IDF:

$$\boxed{\text{documentos} \longrightarrow \text{TF-IDF} \longrightarrow x_1, \ldots, x_n}$$

Ahora podemos aplicar métodos de **aprendizaje no supervisado**:

$$\boxed{\{x_1, \ldots, x_n\} \longrightarrow \text{clusters de documentos.}}$$

El algoritmo no cambia

En aprendizaje no supervisado ya estudiamos problemas de la forma

$$X = \begin{bmatrix} - & - & - & x_1 & - & - & - \\ - & - & - & x_2 & - & - & - \\ & & & \vdots & & & \\ - & - & - & x_n & - & - & - \end{bmatrix}.$$

Para datos tabulares, cada fila representaba una instancia.

Ahora:

cada fila representa un documento

y las características pueden corresponder a:

- frecuencias de palabras;
- valores TF-IDF;
- embeddings.

¿Para qué agrupar textos?

Supongamos que tenemos miles de documentos **sin etiquetas**.

Nos gustaría descubrir automáticamente documentos con características similares.

Por ejemplo:



El algoritmo no conoce previamente el significado de cada grupo.

Similitud entre textos

Dados dos documentos representados por

$$x_i, x_j,$$

podemos comparar sus direcciones mediante

$$\boxed{\cos(x_i, x_j) = \frac{x_i^\top x_j}{\|x_i\| \, \|x_j\|}}$$

Para vectores TF-IDF, los valores son no negativos, por lo que

$$0 \leq \cos(x_i, x_j) \leq 1.$$

$$\text{coseno grande} \quad \Rightarrow \quad \text{documentos similares.}$$

Interpretación geométrica

Dos documentos con vocabulario parecido producen vectores con direcciones similares.

$$\cos(\theta) = \frac{x^\top y}{\|x\|\|y\|}$$

$$\theta \approx 0^\circ \quad \Rightarrow \quad \cos(\theta) \approx 1$$

documentos muy similares

$$\theta \approx 90^\circ \quad \Rightarrow \quad \cos(\theta) \approx 0$$

documentos poco similares

La comparación depende principalmente de la **dirección**, no de la longitud del vector.

De similitud a distancia

Si queremos expresar la relación como una distancia, una opción habitual es

$$\boxed{d_{\cos}(x, y) = 1 - \cos(x, y)}$$

Así:

$$\cos(x, y) \approx 1 \Rightarrow d_{\cos}(x, y) \approx 0,$$

$$\cos(x, y) \approx 0 \Rightarrow d_{\cos}(x, y) \approx 1.$$

Textos más parecidos $\Rightarrow$ menor distancia.

¿Qué hacía K-means?

K-means busca minimizar la suma de las distancias cuadráticas dentro de cada grupo:

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \boldsymbol{\mu}_k\|^2$$

Por tanto, intenta construir clusters donde

los documentos queden cerca de su centroide.

La interpretación del cluster dependerá de las características utilizadas para representar los textos.

Distancia euclidiana y coseno

K-means utiliza normalmente distancia euclidiana.
Pero en texto usamos frecuentemente similitud coseno.

Si normalizamos

$$\|x\| = \|y\| = 1,$$

entonces

$$\begin{aligned}\|x - y\|^2 &= \|x\|^2 + \|y\|^2 - 2x^\top y \\ &= 2 - 2\cos(x, y) \\ &= 2\Big(1 - \cos(x, y)\Big) \\ &= 2d_{\cos}(x, y).\end{aligned}$$

Por tanto, para vectores normalizados:

menor distancia euclidiana $\iff$ mayor similitud coseno.

Spherical K-means

Variante de K-means que utiliza la **similitud coseno**, especialmente útil para agrupar textos.

1. **Normalizar** los vectores: $\|x_i\| = 1$.
2. **Asignar** cada punto al centroide con mayor similitud coseno:

$$z_i = \arg\max_j x_i^T \boldsymbol{\mu}_j.$$

3. **Actualizar** los centroides y normalizarlos:

$$\boldsymbol{\mu}_j = \frac{\sum_{x_i \in C_j} x_i}{\left\| \sum_{x_i \in C_j} x_i \right\|}.$$

Observación

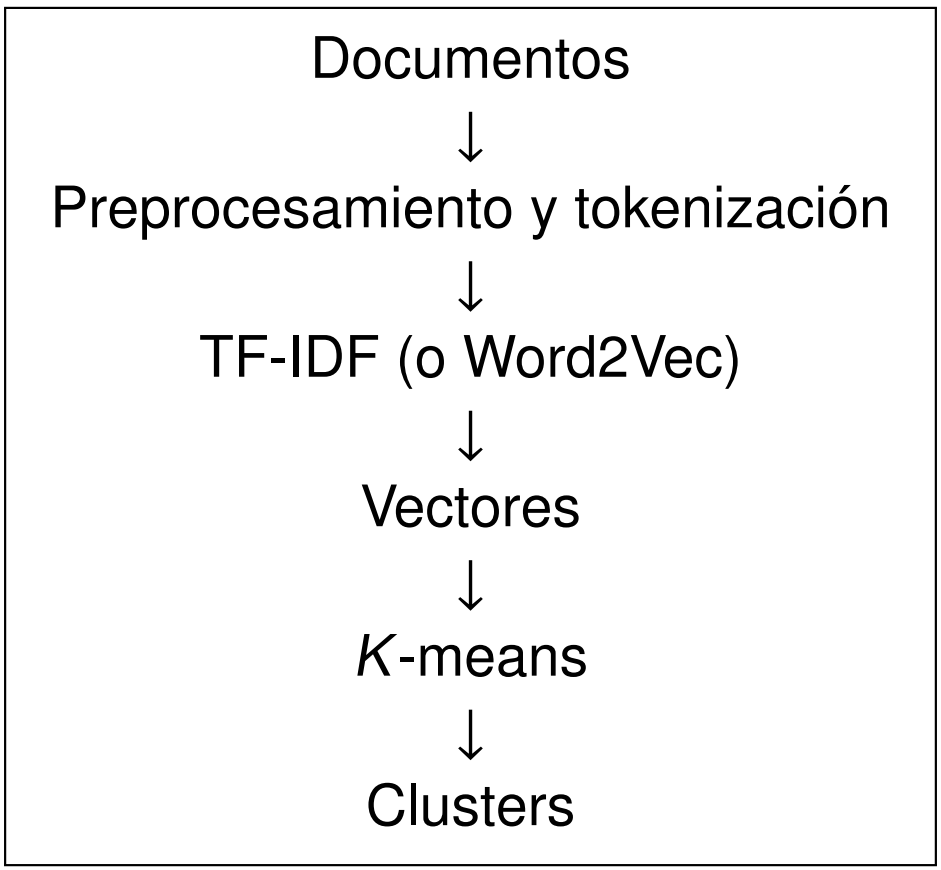
Para vectores unitarios:

$$\|x - \boldsymbol{\mu}\|_2^2 = 2(1 - \cos(x, \boldsymbol{\mu})) = 2d_{\cos}(x, y).$$

Por tanto, ambas distancias producen las mismas asignaciones.

Pipeline para agrupar textos

El procedimiento completo puede escribirse como



Ejemplo conceptual

Supongamos que tenemos noticias sobre

fútbol	acciones	inteligencia artificial
tenis	bancos	redes neuronales
baloncesto	mercados	algoritmos

Después de vectorizar y agrupar podríamos obtener

$$C_1, \quad C_2, \quad C_3.$$

Posteriormente, al inspeccionar los documentos, podríamos interpretar los grupos como

deportes, finanzas, tecnología.

K-means no produce esos nombres.

¿Cómo interpretamos un cluster?

Si utilizamos TF-IDF, cada dimensión corresponde a una palabra.

El centroide

$$\boldsymbol{\mu}_k = [\mu_{k1}, \ldots, \mu_{kp}]$$

también tiene una componente para cada término.

Podemos buscar las componentes más grandes:

$$\boxed{\mu_{kj} \text{ grande} \Rightarrow \text{término característico del cluster}}$$

Por ejemplo:

$$C_1 : \quad \{\text{goal}, \text{match}, \text{team}, \text{player}, \ldots\}$$

puede sugerir un grupo relacionado con deportes.

Implementación con Scikit-Learn

Una implementación básica puede utilizar:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(documents)

model = KMeans(n_clusters=3)

labels = model.fit_predict(X)
```

El resultado

`labels`

indica el cluster asignado a cada documento.

Interpretando los grupos

Podemos inspeccionar las palabras asociadas con las componentes más grandes de cada centroide:

```
terms = vectorizer.get_feature_names_out()

for k, centroid in enumerate(model.cluster_centers_):
    indices = centroid.argsort()[-10:][::-1]
    words = terms[indices]

    print(k, words)
```

Esto puede producir resultados como:

```
0: team game player match goal ...
1: market bank stock price economy ...
2: model data algorithm neural learning ...
```

¿Cuántos clusters utilizar?

K-means requiere especificar previamente

$$K.$$

Las técnicas estudiadas anteriormente continúan siendo aplicables:

- curva de inercia;
- método del codo;
- coeficiente de silueta;
- interpretación de los grupos obtenidos.

La elección de K sigue siendo parte del problema.

Coeficiente de silueta

Mide qué tan bien está agrupado cada punto en su cluster.
Para cada punto x_i, definimos:

- $a(i)$: distancia promedio a los demás puntos de su **mismo cluster**.
- $b(i)$: menor distancia promedio a los puntos de **otro cluster**.

Coeficiente de silueta

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \qquad -1 \leq s(i) \leq 1.$$

- $s(i) \approx 1$: punto bien agrupado.
- $s(i) \approx 0$: punto cerca de la frontera.
- $s(i) < 0$: podría pertenecer a otro cluster.

Definición de $a(i)$ y $b(i)$

Sea C_i el cluster al que pertenece x_i y $d(x_i, x_j)$ una distancia.

Distancia promedio dentro del mismo cluster

$$a(i) = \frac{1}{|C_i| - 1} \sum_{\substack{x_j \in C_i \\ j \neq i}} d(x_i, x_j)$$

Distancia promedio al cluster vecino más cercano

$$b(i) = \min_{C \neq C_i} \left\{ \frac{1}{|C|} \sum_{x_j \in C} d(x_i, x_j) \right\}$$

- $a(i)$ pequeño: buena cohesión dentro del cluster.
- $b(i)$ grande: buena separación entre clusters.

Nota: Si $|C_i| = 1$, se define $s(i) = 0$.

Elegir K mediante el coeficiente de silueta

Para cada valor de K, calculamos la **silueta promedio**:

$$S(K) = \frac{1}{N} \sum_{i=1}^{N} s(i).$$

Criterio: elegir el K con mayor silueta promedio.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

for k in range(2, 9):
    modelo = KMeans(n_clusters=k,n_init=10)
    etiquetas = modelo.fit_predict(X)

    score = silhouette_score(X, etiquetas)
    print(f"K={k}, silueta={score:.3f}")
```

Importante

El mayor coeficiente de silueta no siempre corresponde al número de grupos más interpretable.

La representación cambia los clusters

Consideremos los mismos documentos.

Podemos aplicar K-means sobre

$$X_{\text{BoW}}, \qquad X_{\text{TF-IDF}}, \qquad X_{\text{embedding}}.$$

Aunque el algoritmo sea el mismo,

las geometrías del espacio son diferentes

y por tanto pueden obtenerse agrupamientos distintos.

La representación forma parte fundamental del modelo.

TF-IDF o embeddings

	TF-IDF	**Embeddings**
Tipo	Disperso	Denso
Dimensiones	Palabras	Latentes
Similitud	Léxica	Puede capturar semántica
Interpretación	Más directa	Menos directa

Con TF-IDF, dos documentos son similares principalmente si comparten términos.

Con embeddings, documentos con vocabulario distinto pueden quedar próximos si sus representaciones capturan información semántica.

¿Qué significan los clusters?

Un cluster no es necesariamente una categoría “verdadera” del conjunto de datos.

El resultado depende de

preprocesamiento
representación
medida geométrica
K
algoritmo

Por eso es necesario

inspeccionar e interpretar los resultados.

Práctica #6: agrupamiento de documentos

Objetivo

Agrupar documentos según su contenido utilizando **TF-IDF y K-means**, sin conocer previamente las categorías de los textos.

1. Representar los documentos mediante TF-IDF.
2. Aplicar K-means para distintos valores de K.
3. Evaluar los agrupamientos mediante inercia y coeficiente de silueta.
4. Identificar las palabras más importantes de cada centroide.
5. Interpretar los temas encontrados.

Pregunta: ¿Podemos descubrir automáticamente los temas de una colección de documentos?

Dataset: 20 Newsgroups

Utilizaremos una colección de mensajes de discusión sobre diferentes temas.

Para simplificar, seleccionamos cuatro categorías:

- Ciencia espacial.
- Béisbol.
- Gráficos por computadora.
- Política de Medio Oriente.

Importante

Las categorías originales no se utilizan para entrenar K-means.
El objetivo es descubrir grupos utilizando únicamente el contenido de los documentos.

¿Cuántos grupos esperamos encontrar?

Paso 1: cargar los documentos

```
from sklearn.datasets import fetch_20newsgroups

categorias = ["sci.space","rec.sport.baseball", "comp.graphics","talk.
    politics.mideast"]

datos = fetch_20newsgroups(
    subset="train",
    categories=categorias,
    remove=("headers", "footers", "quotes"),
    shuffle=True,
    random_state=42
)

documentos = datos.data

print("Documentos:", len(documentos))
print(documentos[11][:500])
```

Paso 2: representación mediante TF-IDF

Transformamos cada documento en un vector numérico:

$$d_i \longrightarrow x_i = (x_{i1}, x_{i2}, \ldots, x_{ip}).$$

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizador = TfidfVectorizer(stop_words="english",min_df=5,max_df=0.8,
     max_features=5000,norm="l2")

X = vectorizador.fit_transform(documentos)

print(X.shape)
```

Observaciones

- Cada fila representa un documento.
- Cada columna representa un término.
- Los vectores están normalizados: $\|x_i\|_2 = 1$ para documentos no vacíos.

Parámetros de `TfidfVectorizer`

- `stop_words="english"`: elimina palabras comunes del inglés, como *the*, *and*, *is*.
- `min_df=5`: conserva términos que aparecen en al menos **5 documentos**.
- `max_df=0.8`: elimina términos que aparecen en más del **80 % de los documentos**.
- `max_features=5000`: limita el vocabulario a los **5000 términos más frecuentes** que cumplen los filtros anteriores.
- `norm="l2"`: normaliza cada vector TF-IDF no nulo:

$$\widehat{x_i} = \frac{x_i}{\|x_i\|_2}, \qquad \|\widehat{x_i}\|_2 = 1.$$

Objetivo

Reducir términos poco informativos, controlar la dimensionalidad y obtener vectores comparables.

Paso 3: aplicar K-means

Aplicamos K-means para diferentes números de grupos, $K = 2, 3, \ldots, 8$.

```
from sklearn.cluster import KMeans

modelos = {}

for k in range(2, 9):
    modelo = KMeans(n_clusters=k,random_state=42,n_init=10)

    modelo.fit(X)
    modelos[k] = modelo

    print(k, modelo.inertia_)
```

Recordatorio

K-means clásico minimiza la suma de distancias euclidianas al cuadrado:

$$J = \sum_{i=1}^{N} \|x_i - \boldsymbol{\mu}_{z_i}\|^2.$$

Paso 4: evaluar los agrupamientos

Compararemos dos criterios:

- **Inercia:** queremos valores pequeños.
- **Silueta:** preferimos valores cercanos a 1.

```
from sklearn.metrics import silhouette_score

inercias = []
siluetas = []

for k, modelo in modelos.items():
    inercias.append(modelo.inertia_)

    s = silhouette_score(X,modelo.labels_,metric="euclidean",sample_size=
         min(1000, X.shape[0]),random_state=42)

    siluetas.append(s)
```

Pregunta

¿El valor de K con mejor silueta coincide con el sugerido por el método del codo?

Paso 5: visualizar los resultados

```
import matplotlib.pyplot as plt

valores_k = list(modelos.keys())

plt.figure()
plt.plot(valores_k, inercias, marker="o")
plt.xlabel("Numero de clusters K")
plt.ylabel("Inercia")
plt.title("Metodo del codo")
plt.show()

plt.figure()
plt.plot(valores_k, siluetas, marker="o")
plt.xlabel("Numero de clusters K")
plt.ylabel("Silueta")
plt.title("Coeficiente de silueta")
plt.show()
```

Discusión

La inercia generalmente disminuye al aumentar K. Por eso no basta con elegir el valor que produce la menor inercia.

Paso 6: interpretar los clusters

Seleccionamos un valor de K y examinamos las palabras con mayor peso en cada centroide.

```
import numpy as np

modelo = modelos[4]

terminos = vectorizador.get_feature_names_out()

for j, centroide in enumerate(modelo.cluster_centers_):
    indices = np.argsort(centroide)[::-1][:10]
    palabras = terminos[indices]

    print(f"Cluster {j}:")
    print(", ".join(palabras))
```

Preguntas

1. ¿Qué tema parece representar cada cluster?
2. ¿Qué palabras permiten identificarlo?
3. ¿Existen clusters que mezclen varios temas?

Paso 7: comparar con las categorías reales

Ahora revelamos las categorías originales y comparamos los resultados.

```
import pandas as pd

etiquetas_reales = [
    datos.target_names[i]
    for i in datos.target
]

tabla = pd.crosstab( pd.Series(modelo.labels_, name="Cluster"),pd.Series(
     etiquetas_reales, name="Tema real") )

print(tabla)
```

Preguntas

- ¿Los clusters coinciden con los temas originales?
- ¿Qué categorías se confunden más?
- ¿Por qué los identificadores de los clusters no tienen que coincidir con las etiquetas?

Práctica #6: modificar la representación

Entrega: Martes 20 de Octubre.

Nuevo experimento

Repetir el agrupamiento sustituyendo TF-IDF por otra representación de los documentos.

1. Utilizar `CountVectorizer` para representar frecuencias de términos.
2. Normalizar los vectores mediante norma L_2.
3. Aplicar nuevamente K-means.
4. Comparar la silueta, la curva de inercia y la interpretación de los clusters.

Preguntas finales

¿La representación de los documentos influye en los grupos encontrados?

¿Podríamos utilizar embeddings en lugar de TF-IDF?

Resumen

El agrupamiento de textos requiere dos pasos fundamentales:

$$\boxed{\text{Texto} \longrightarrow \text{representación vectorial} \longrightarrow \text{clustering}}$$

Por ejemplo:

$$\boxed{\text{documentos} \rightarrow \text{TF-IDF} \rightarrow K\text{-means} \rightarrow \text{clusters}}$$

Las ideas principales son:

- cada documento se representa como un vector;
- la similitud coseno permite comparar vectores de texto;
- K-means agrupa vectores alrededor de centroides;
- los clusters deben interpretarse posteriormente.

Siguiente paso: clasificación de textos

Hasta ahora:

documentos sin etiquetas $\longrightarrow$ clusters

¿Qué ocurre si conocemos una etiqueta para cada documento?

reseña positiva reseña negativa	o	deportes finanzas tecnología

Entonces podemos utilizar **aprendizaje supervisado**.

¡Gracias!